
Programmation orientée objet

TD n° 1

Exercice 1. Classes, attributs et méthodes

Q- 1.1 Classes

L'objectif de ce premier exercice est de se familiariser avec les **classes**, les **attributs** et les **méthodes**.

Nous ne pourrons donc utiliser **AUCUNE** librairie.

Dans cet exercice, nous allons développer une librairie permettant de manipuler des objets géométriques simples en deux dimensions (des points, segments, des polygones et des cercles).

Q- 1.2 Première Classe Point

Dans un premier temps, nous allons réaliser une classe `Point` qui possédera 2 attributs principaux : deux flottants en double précision (`double`), les coordonnées `x` et `y` du point. Un utilisateur de la classe devra pouvoir créer des points, les afficher à l'aide d'une fonction `print`, calculer la distance entre deux points avec une fonction `distance` et point milieu de deux points avec une fonction `center`.

Créer la classe `Point` ainsi qu'un petit programme de test permettant d'afficher les valeurs de 3 points, une distance et un point milieu. Nous utiliserons trois fichiers : `Point.h`, `Point.cpp` et `main.cpp` ainsi qu'un fichier `Makefile`.

Q- 1.3 Liste de points

Nous souhaitons permettre à l'utilisateur de notre programme de créer des objets géométriques et de maintenir une liste de ses objets.

Dans un premier temps créer une liste qui ne contiendra que des objets de type `Point`. La classe `ListP` permettra de créer et de maintenir une liste simplement chaînée de `Point` avec les méthodes suivantes :

- ▶ `isempty(...)` qui retourne un booléen `True` si la liste est vide,
- ▶ `push_pos(int pos, Point p)` qui insère le point `p` en position `pos` si la position existe et en queue de liste sinon,
- ▶ `delete_pos(int pos)` qui supprime le point en position `pos` si la position existe et supprime le dernier sinon,
- ▶ `size(...)` qui renvoie la taille de la liste,
- ▶ `print(...)`, permettant d'afficher le contenu de la liste.

Q- 1.4 Test de listes

Écrire un programme de test permettant à l'utilisateur de gérer une liste de points à l'aide d'un menu en mode texte avec des options pour saisir et ajouter des points à la liste, supprimer des points dans la liste et afficher la liste.

Q- 1.5 Des points uniques

Nous souhaitons donner à chaque nouveau point créer un identifiant unique `uid` qui sera un entier non-signé (`unsigned int`). Modifier la classe `Point` pour prendre en compte cet identifiant et pour faire en sorte que cet identifiant soit affiché chaque fois qu'un point est affiché.